

STRUTTURE DATI FONDAMENTALI: ANALISI, IMPLEMENTAZIONE IN PYTHON E APPLICAZIONI REALI

Dagli Array ai Grafi: complessità computazionale, algoritmi fondamentali e casi d'uso nei sistemi informatici reali

Antonio Adinolfi

Docente di Informatica – Educational Technical Paper

Abstract – Le strutture dati costituiscono l'infrastruttura concettuale su cui si fonda ogni algoritmo: non esiste procedura computazionale efficiente che prescindano dal modo in cui i dati sono organizzati, indicizzati e collegati in memoria. Questo articolo presenta un'analisi tecnico-didattica delle sette strutture dati fondamentali dell'informatica – array, liste concatenate, pile, code, tabelle hash, alberi e grafi – affrontandole secondo un duplice livello di lettura: una descrizione intuitiva basata su analogie concrete e una formalizzazione rigorosa basata sulla notazione asintotica di Landau (Big-O). Per ciascuna struttura viene mantenuta una distinzione esplicita tra il modello teorico astratto, le sue implementazioni concrete e i possibili ambiti applicativi, evitando equivalenze assolute tra concetti teorici e tipi nativi di un particolare linguaggio. Ogni struttura è corredata da un'implementazione completa in Python 3, da esempi eseguibili, dall'analisi della complessità nei casi migliore, medio e peggiore, e da applicazioni reali tratte da sistemi informatici concreti. L'articolo introduce inoltre l'algoritmo di Dijkstra per il calcolo del cammino minimo su grafi pesati, una sezione dedicata ai criteri di scelta della struttura dati più adatta a un dato problema, e un caso di studio integrato che combina più strutture in un unico sistema simulato. L'obiettivo è fornire uno strumento di riferimento che accompagni il lettore dal concetto astratto all'implementazione concreta, passando per l'analisi della complessità e l'applicazione reale, con un rigore scientifico compatibile con un vero articolo tecnico-didattico in stile IEEE.

Parole chiave – Data Structures, Algorithms, Python, Computational Complexity, Big-O, Array, Linked List, Stack, Queue, Hash Table, Tree, Graph, Dijkstra.

I. INTRODUZIONE

Un algoritmo non opera mai nel vuoto: agisce sempre su dati organizzati secondo una particolare *struttura dati*, ed è proprio questa organizzazione a determinare quanto efficientemente l'algoritmo possa muoversi, cercare, inserire o rimuovere informazioni. Una struttura dati non è definita soltanto dal modo in cui i dati sono disposti in memoria, ma anche dall'insieme delle operazioni che essa rende efficienti: un array rende efficiente l'accesso per indice, una lista concatenata rende efficiente l'inserimento in testa, una tabella hash rende efficiente la ricerca per chiave. Il rapporto tra dati e procedure è riassunto dal principio, ormai classico nella letteratura informatica [1, 2, 3, 4]:

“Data structures + algorithms = efficient software”

Si consideri un esempio elementare: cercare un elemento in una collezione di n valori. Se i dati sono memorizzati in un array non ordinato, la ricerca nel caso peggiore richiede di esaminare tutti gli n elementi. Se gli stessi dati sono organizzati in un array ordinato e si applica la ricerca binaria, il numero di confronti nel caso peggiore scende a $O(\log n)$. Se infine i dati sono indicizzati in una tabella hash, la ricerca ha costo medio $O(1)$. Stesso problema, stessi dati, tre strutture dati diverse, tre comportamenti asintotici radicalmente differenti.

È importante, fin da questa introduzione, distinguere con cura tre livelli concettuali che l'intero articolo manterrà separati:

A) la struttura dati astratta, ossia il modello teorico (es. “coda”, “tabella hash”);

- B) l'implementazione concreta** che un linguaggio o una libreria fornisce di quel modello (es. `collections.deque` come implementazione efficiente del modello di coda);
- C) l'applicazione** in cui quella struttura trova impiego (es. una coda applicata alla gestione delle richieste di un server).

Confondere questi tre livelli – ad esempio affermando in modo assoluto che “una coda è un deque”, oppure che “un albero è un file system” – è un errore concettuale diffuso, discusso più approfonditamente in Sezione XV. L'obiettivo di questo articolo è fornire una trattazione rigorosa delle sette strutture dati fondamentali – array, liste concatenate, pile, code, tabelle hash, alberi e grafi – ancorandola a implementazioni Python reali ed eseguibili, e a un algoritmo di riferimento sui grafi (Dijkstra) di rilevanza pratica primaria. Il percorso didattico proposto segue sempre la stessa progressione: *struttura dati* → *operazioni* → *algoritmi* → *complessità computazionale* → *implementazione Python* → *applicazione reale*.

II. COMPLESSITÀ COMPUTAZIONALE E NOTAZIONE BIG-O

La notazione asintotica $O(\cdot)$ (Big-O) descrive il **limite superiore asintotico** della crescita del costo computazionale di un algoritmo al crescere della dimensione n dell'input [1, 3]. È importante correggere immediatamente un equivoco molto diffuso: **Big-O non è sinonimo di “caso peggiore”**. La notazione $O(\cdot)$ è uno strumento matematico che può essere applicato a qualunque scenario di analisi – caso migliore, medio o peggiore – e tale scenario deve sempre essere dichiarato esplicitamente insieme alla notazione. Ad esempio, è corretto

Tabella 1: Classi di complessità asintotica più frequenti.

Notazione	Descrizione / esempio tipico
$O(1)$	Tempo costante: accesso ad array per indice
$O(\log n)$	Dimezzamento a ogni passo: ricerca binaria
$O(n)$	Scansione lineare: ricerca in lista/array
$O(n \log n)$	Algoritmi divide-et-impera: merge sort
$O(n^2)$	Doppio ciclo annidato: bubble sort, matrici

scrivere che la ricerca lineare ha caso migliore $O(1)$ e caso peggiore $O(n)$: in entrambe le affermazioni si utilizza la notazione Big-O, riferita però a due scenari distinti.

Occorre inoltre distinguere due dimensioni di costo indipendenti:

- **Time complexity:** numero di operazioni elementari eseguite in funzione di n ;
- **Space complexity:** quantità di memoria aggiuntiva richiesta in funzione di n , spesso trascurata ma altrettanto rilevante (Sezione XV).

e tre scenari di analisi:

- **Best case:** costo nella configurazione di input più favorevole;
- **Average case:** costo atteso su input tipici, spesso calcolato sotto ipotesi probabilistiche sulla distribuzione dei dati;
- **Worst case:** costo massimo su qualunque input di dimensione n .

La Tabella 1 riassume le classi di complessità asintotica più ricorrenti nelle strutture dati trattate in questo articolo.

Un esempio didattico classico che illustra la necessità di dichiarare esplicitamente lo scenario di analisi è il confronto tra ricerca lineare e ricerca binaria (Listato 1, Tabella 2). Si noti che la ricerca binaria **richiede che l'array sia già ordinato**: la sua complessità logaritmica non è valida su dati non ordinati.

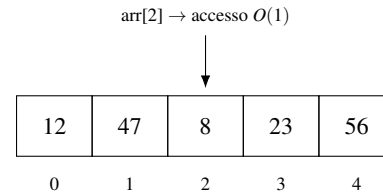
```

1 def ricerca_lineare(arr, target):
2     """Scansiona l'array elemento per
3     elemento, senza richiedere un
4     ordinamento preventivo."""
5     for i, val in enumerate(arr):
6         if val == target:
7             return i
8     return -1
9
10 def ricerca_binaria(arr, target):
11     """Richiede arr ORDINATO.
12     Dimezza l'intervallo di ricerca
13     ad ogni iterazione."""
14     sx, dx = 0, len(arr) - 1
15     while sx <= dx:
16         mid = (sx + dx) // 2
17         if arr[mid] == target:
18             return mid
19         elif arr[mid] < target:
20             sx = mid + 1
21         else:
22             dx = mid - 1
23     return -1

```

Listato 1: Ricerca lineare e ricerca binaria (quest'ultima richiede un array ordinato).**Tabella 2:** Complessità di ricerca lineare e binaria per scenario.

Algoritmo	Best case	Worst case
Ricerca lineare	$O(1)$	$O(n)$ (average: $O(n)$)
Ricerca binaria	$O(1)$	$O(\log n)$

**Fig. 1:** Array e accesso tramite indice.

Su un array di un milione di elementi ordinati, la ricerca lineare esegue nel caso peggiore un milione di confronti, mentre la ricerca binaria ne esegue al più venti: la differenza tra $O(n)$ e $O(\log n)$ è quindi misurabile in ordini di grandezza, non un dettaglio accademico.

III. ARRAY E SEQUENZE PYTHON

Un *array* è, dal punto di vista teorico, una collezione di elementi omogenei memorizzati in un blocco di memoria **contiguo**, ciascuno raggiungibile in tempo costante tramite un *indice* numerico: l'indirizzo di memoria dell'elemento i -esimo si calcola come $\text{base} + i \times \text{dimensione_elemento}$, formula aritmetica calcolabile in tempo costante che garantisce l'accesso $O(1)$.

Come mostrato in Fig. 1, il modello dell'array statico è distinto dal *dynamic array*, che si ridimensiona automaticamente quando lo spazio allocato si esaurisce. **Il tipo list di Python è un'implementazione concreta del modello di dynamic array**, non un array nel senso classico dei linguaggi C/C++: *list* è una sequenza dinamica che, nell'implementazione CPython, utilizza un'area contigua di riferimenti agli oggetti e una gestione dinamica della capacità, sovra-allocando periodicamente spazio aggiuntivo per evitare una riallocazione a ogni singolo inserimento [8, 5]. Questo è ciò che si intende con **costo ammortizzato**: la maggior parte delle operazioni di *append* costa $O(1)$ perché trova spazio già disponibile, mentre occasionalmente (quando la capacità si esaurisce) l'operazione innesca una riallocazione e una copia dell'intero contenuto, di costo $O(n)$; ripartendo però questo costo occasionale su tutte le operazioni precedenti, il costo *medio per operazione*, calcolato su una sequenza di n *append*, resta $O(1)$.

La Tabella 3 riassume la complessità delle operazioni fondamentali su *list*, in accordo con la documentazione ufficiale e con la Complexity Wiki di Python [5, 8].

Il Listato 2 mostra le operazioni fondamentali su *list*.

```

1 buffer = [12, 47, 8, 23, 56]
2
3 # Accesso per indice: O(1)
4 elemento = buffer[2]           # 8
5
6 # Ricerca: O(n) nel caso peggiore

```

Tabella 3: Complessità delle operazioni su list (dynamic array).

Operazione	Costo	Note
Accesso per indice	$O(1)$	Aritmetica di indirizzo
Ricerca (non ordinato)	$O(n)$	Scansione lineare
append in coda	$O(1)$	Ammortizzato
Inserimento in testa/mezzo	$O(n)$	Shift degli elementi
Rimozione in coda	$O(1)$	–
Rimozione in testa/mezzo	$O(n)$	Shift degli elementi

```

7 trovato = 23 in buffer # True
8
9 # Inserimento in coda:  $O(1)$  ammortizzato
10 buffer.append(99)
11
12 # Inserimento in posizione
13 # arbitraria:  $O(n)$ , per lo shift
14 buffer.insert(1, 0)
15
16 # Rimozione in coda:  $O(1)$ 
17 buffer.pop()
18
19 # Rimozione in posizione
20 # arbitraria:  $O(n)$ 
21 buffer.pop(1)
22
23 # Attraversamento:  $O(n)$ 
24 for v in buffer:
25     pass

```

Listato 2: Operazioni fondamentali su list (dynamic array).

Applicazioni reali: gli array trovano applicazione nella rappresentazione di immagini digitali come matrici di pixel e nel calcolo numerico, dove librerie come NumPy realizzano array contigui a basso livello per operazioni vettoriali e matriciali ad alte prestazioni [10]; sono inoltre alla base dei buffer di I/O.

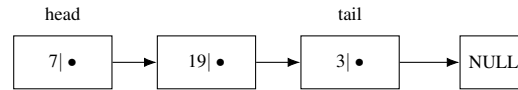
A. Tuple

Va corretta un'ulteriore semplificazione frequente: tuple non è semplicemente “un array statico”. Più correttamente, tuple è una **sequenza immutabile**: nell'implementazione CPython i suoi elementi sono memorizzati mediante riferimenti in una struttura contigua, analogamente a list, ma la struttura non può essere modificata dopo la creazione (né in lunghezza né nei riferimenti contenuti). L'analogia con un array statico può essere utile a scopo puramente didattico per fissare l'idea dell'immutabilità e della dimensione fissa, mantenendo però la consapevolezza che si tratta di un'analogia, non di un'identità strutturale con l'array statico dei linguaggi a basso livello. Grazie all'immutabilità, una tuple composta da soli elementi hashabili è a sua volta hashabile, e può quindi essere usata come chiave in un dict o come elemento di un set – un impiego precluso a list.

IV. LINKED LIST

Una *linked list* (lista concatenata) è una struttura dinamica composta da *nodi*, ciascuno contenente un dato e un riferimento (next) al nodo successivo. A differenza dell'array, i nodi non occupano memoria contigua: la struttura è tenuta insieme esclusivamente dai riferimenti. Il primo nodo è detto

head, l'ultimo *tail* (il cui riferimento successivo è None).

**Fig. 2:** Struttura di una Linked List (singly linked).

Come illustrato in Fig. 2, il Listato 3 implementa una singly linked list con le operazioni fondamentali.

```

1 class Nodo:
2     """Un nodo della lista concatenata."""
3     def __init__(self, dato):
4         self.dato = dato
5         self.successivo = None
6
7 class ListaConcatenata:
8     """Singly linked list con le
9     operazioni fondamentali."""
10    def __init__(self):
11        self.head = None
12
13    def append(self, dato):
14        """Inserimento in coda. Poiché
15        non è mantenuto un riferimento
16        a tail, richiede prima di
17        attraversare la lista:  $O(n)$ ."""
18        nuovo = Nodo(dato)
19        if self.head is None:
20            self.head = nuovo
21            return
22        corrente = self.head
23        while corrente.successivo:
24            corrente = corrente.successivo
25        corrente.successivo = nuovo
26
27    def prepend(self, dato):
28        """Inserimento in testa:  $O(1)$ ,
29        poiché il riferimento alla
30        testa è sempre disponibile."""
31        nuovo = Nodo(dato)
32        nuovo.successivo = self.head
33        self.head = nuovo
34
35    def insert(self, indice, dato):
36        """Inserimento in posizione
37        arbitraria: la LOCALIZZAZIONE
38        della posizione costa  $O(n)$ ;
39        l'inserimento vero e proprio,
40        una volta individuato il nodo
41        precedente, costa  $O(1)$ ."""
42        if indice == 0:
43            return self.prepend(dato)
44        nuovo = Nodo(dato)
45        corrente = self.head
46        for _ in range(indice - 1):
47            corrente = corrente.successivo
48        nuovo.successivo = corrente.successivo
49        corrente.successivo = nuovo
50
51    def delete(self, dato):
52        """Cancellazione per valore: la
53        RICERCA del nodo costa  $O(n)$ ;
54        la rimozione stessa, una volta
55        trovato il nodo, costa  $O(1)$ ."""
56        if self.head is None:
57            return
58        if self.head.dato == dato:
59            self.head = self.head.successivo
60            return
61        corrente = self.head
62        while corrente.successivo:
63            if corrente.successivo.dato == dato:
64                corrente.successivo = \
65                    corrente.successivo.successivo
66                return
67        corrente = corrente.successivo
68
69    def search(self, dato):
70        """Ricerca:  $O(n)$ ."""
71        corrente = self.head

```

```

72 while corrente:
73     if corrente.data == dato:
74         return True
75     corrente = corrente.successivo
76 return False
77
78 def display(self):
79     """Attraversamento: O(n)."""
80     valori = []
81     corrente = self.head
82     while corrente:
83         valori.append(str(corrente.data))
84         corrente = corrente.successivo
85     return " -> ".join(valori)

```

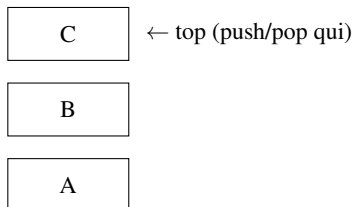
Listato 3: Implementazione di una Singly Linked List.

Va sottolineato un punto spesso trascurato: affermare semplicemente “inserimento $O(1)$ ” o “cancellazione $O(1)$ ” per una linked list è impreciso senza specificare la condizione. **Se il riferimento al nodo (o alla posizione) è già disponibile**, inserimento e cancellazione in quel punto costano $O(1)$, perché richiedono solo la modifica di uno o due riferimenti. **Se invece il nodo deve prima essere individuato** (ad esempio per valore, come in `delete`), la ricerca preliminare costa $O(n)$, e questo costo domina la complessità complessiva dell’operazione.

Si distinguono tre varianti: la *singly linked list* (ogni nodo punta solo al successivo), la *doubly linked list* (ogni nodo mantiene anche un riferimento al predecessore, consentendo attraversamento bidirezionale) e la *circular linked list* (l’ultimo nodo punta nuovamente al primo). **Applicazioni reali:** gestione di sequenze dinamiche la cui dimensione non è nota a priori, e implementazione di strutture a doppia estremità efficienti (Sezione VI).

V. STACK

Uno *stack* (pila) è una struttura dati astratta che rispetta il principio **LIFO** (Last In, First Out): l’ultimo elemento inserito è il primo a essere rimosso, come in una pila di piatti.



LIFO: Last In, First Out

Fig. 3: Funzionamento LIFO di uno Stack.

Come illustrato in Fig. 3, le operazioni fondamentali sono `push` (inserimento in cima), `pop` (rimozione dalla cima) e `peek` (lettura senza rimozione). In Python il modello di stack può essere implementato in modo efficiente utilizzando `list.append()` per `push` e `list.pop()` (senza argomenti) per `pop`, entrambe operazioni sull’estremità finale della lista. La ricerca di un elemento generico all’interno dello stack, se necessaria, richiede una scansione completa e costa $O(n)$: lo stack non è però concepito per supportare ricerche efficienti, essendo ottimizzato esclusivamente per l’accesso alla cima. Il Listato 4 mostra due implementazioni equivalenti.

```

1 # A) Stack con list nativa
2 stack = []
3 stack.append(10)      # push: O(1) ammortizzato
4 stack.append(20)
5 stack.append(30)
6 cima = stack.pop()    # pop: O(1) -> 30
7
8 # B) Classe Stack dedicata
9 class Stack:
10     def __init__(self):
11         self._dati = []
12
13     def push(self, valore):
14         """O(1) ammortizzato."""
15         self._dati.append(valore)
16
17     def pop(self):
18         """O(1)."""
19         if self.is_empty():
20             raise IndexError("stack vuoto")
21         return self._dati.pop()
22
23     def peek(self):
24         """O(1)."""
25         if self.is_empty():
26             raise IndexError("stack vuoto")
27         return self._dati[-1]
28
29     def is_empty(self):
30         """O(1)."""
31         return len(self._dati) == 0

```

Listato 4: Stack tramite list e tramite classe dedicata.

Il Listato 5 applica lo stack a un problema classico: verificare se una sequenza di parentesi è bilanciata.

```

1 def parentesi_bilanciate(espressione):
2     """Verifica il bilanciamento di
3     (), [], {} usando uno stack.
4     Costo: O(n)."""
5     coppie = {'(': ')', '[': ']', '{': '}'}
6     stack = []
7     for carattere in espressione:
8         if carattere in "([{":
9             stack.append(carattere)
10        elif carattere in ")]}":
11            if not stack or \
12               stack.pop() != coppie[carattere]:
13                return False
14        return len(stack) == 0
15
16 print(parentesi_bilanciate("[(a+b)*(c-d)]")) # True
17 print(parentesi_bilanciate("[(a+b)]"))       # False

```

Listato 5: Verifica di parentesi bilanciate tramite Stack.

Applicazioni reali: il modello dello stack regge la *call stack* di ogni linguaggio di programmazione, ovvero il meccanismo con cui vengono gestite le chiamate di funzione annidate e ricorsive; è alla base delle funzionalità di *undo/redo* negli editor di testo; trova impiego nel *parsing delle espressioni* nei compilatori; è la struttura ausiliaria dell’algoritmo di visita in profondità (*DFS*, Sezione IX); e regge concettualmente la cronologia di navigazione “indietro” dei browser web.

VI. QUEUE

Una *queue* (coda) è il modello astratto che rispetta il principio opposto allo stack: **FIFO** (First In, First Out). Il primo elemento inserito è il primo a essere rimosso.

Un errore implementativo comune consiste nell’usare `list.pop(0)` per rimuovere l’elemento in testa a una list: poiché list è un dynamic array (Sezione III), rimuovere il primo elemento richiede di traslare tutti gli $n - 1$ elementi

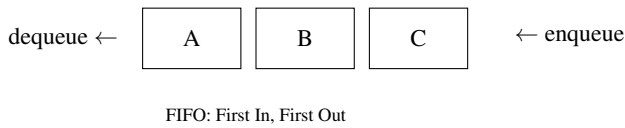


Fig. 4: Funzionamento FIFO di una Queue.

rimanenti verso l’inizio del blocco contiguo, con costo $O(n)$ per ogni singola operazione.

Una struttura concreta più adatta a implementare efficientemente il modello di coda è `collections.deque`. È importante non identificare deque con una semplice linked list: `collections.deque` è una struttura a doppia estremità progettata per rendere efficienti le operazioni di inserimento e rimozione a entrambe le estremità; nell’implementazione CPython viene utilizzata internamente una sequenza di blocchi collegati, una scelta implementativa diversa sia da un singolo blocco contiguo (come in `list`) sia da una linked list nodo-per-nodo [6]. Su deque, le quattro operazioni alle estremità – `append`, `appendleft`, `pop`, `popleft` – costano tutte $O(1)$. Il Listato 6 confronta l’anti-pattern con la soluzione corretta.

```
1 from collections import deque
2
3 # ANTI-PATTERN: O(n) per ogni dequeue
4 coda_lenta = [10, 20, 30]
5 primo = coda_lenta.pop(0) # O(n): shift!
6
7 # SOLUZIONE: deque, O(1) su entrambe
8 # le estremità
9 coda = deque()
10 coda.append(10) # enqueue (coda): O(1)
11 coda.append(20)
12 coda.appendleft(5) # inserimento in testa: O(1)
13 primo = coda.popleft() # dequeue (testa): O(1)
14 ultimo = coda.pop() # rimozione in coda: O(1)
15 vuota = len(coda) == 0 # is_empty: O(1)
```

Listato 6: Queue: l’errore di `list.pop(0)` e la soluzione con `deque`.

Applicazioni reali: il modello di coda è naturale per le code di stampa e per lo scheduling dei processi nei sistemi operativi; sistemi di messaggistica come Apache Kafka [11] e RabbitMQ [12] realizzano, a un livello architetturale più ampio, code persistenti e distribuite per la comunicazione asincrona tra servizi; una coda è inoltre la struttura ausiliaria dell’algoritmo di visita in ampiezza (*BFS*, Sezione IX). Il Listato 7 simula una coda di richieste in attesa su un server.

```
1 from collections import deque
2
3 richieste = deque()
4 richieste.append("GET /home")
5 richieste.append("POST /login")
6 richieste.append("GET /profilo")
7
8 print(f"In coda: {len(richieste)}")
9 while richieste:
10     corrente = richieste.popleft()
11     print(f"Elabora: {corrente}")
```

Listato 7: Simulazione di una coda di richieste server.

VII. HASH TABLE

Una *hash table* (tabella hash) è il modello teorico che associa a ogni *chiave* (key) un *valore* (value) tramite una *funzione hash*, che trasforma la chiave in un indice numerico (*bucket*) all’interno di un array sottostante. Quando due chiavi distinte producono lo stesso indice si parla di *collisione*. La teoria distingue almeno due famiglie di strategie per gestire le collisioni:

- **chaining** (concatenamento): ogni bucket mantiene una struttura secondaria (tipicamente una lista) di tutte le coppie chiave-valore che vi collidono;
- **open addressing** (indirizzamento aperto): in caso di collisione, si cerca un altro slot libero nell’array stesso, secondo una sequenza di probing predefinita.

È fondamentale distinguere questo **modello teorico** dall’**implementazione concreta** fornita da un linguaggio. Il dizionario Python (`dict`) è un’implementazione concreta basata su hashing del paradigma di hash table, ma l’implementazione di `dict` in CPython utilizza una struttura interna basata su hashing e una specifica strategia di gestione delle collisioni (storicamente una forma di indirizzamento aperto con probing pseudo-casuale); pertanto `dict` non deve essere identificato semplicemente con la tecnica di “concatenamento nei bucket”, né si deve assumere che i dettagli implementativi di CPython restino invariati tra versioni del linguaggio [8, 5].

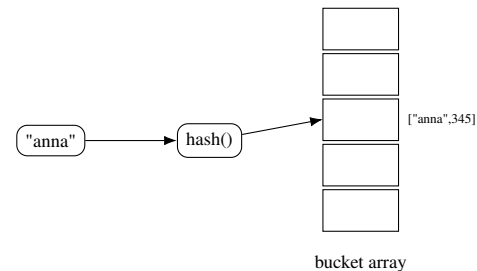


Fig. 5: Mapping key → hash → bucket (modello teorico generale).

Indipendentemente dalla strategia di gestione delle collisioni, il comportamento atteso di una tabella hash ben dimensionata è: ricerca, inserimento e cancellazione in tempo **medio** $O(1)$, e in tempo **peggiore teorico** $O(n)$ qualora un numero elevato di chiavi collida nello stesso bucket. È importante comprendere che “complessità media $O(1)$ ” non significa “costo sempre costante”: significa che, sotto ipotesi ragionevoli sulla distribuzione delle chiavi e sul fattore di carico della tabella, il costo atteso per operazione non cresce con n ; singole operazioni possono comunque costare di più in presenza di collisioni o durante un ridimensionamento interno della struttura. Il Listato 8 mostra tre applicazioni tipiche del modello.

```
1 # 1) Rubrica telefonica: lookup O(1) medio
2 rubrica = {
3     "Anna": "345-1122334",
4     "Marco": "347-9988776",
5 }
6 numero = rubrica.get("Anna")
```

```

7
8 # 2) Conteggio delle frequenze delle parole
9 def conta_frequenze(testo):
10     """Conta quante volte compare ogni
11     parola. Costo: O(n)."""
12     frequenze = {}
13     for parola in testo.lower().split():
14         frequenze[parola] = \
15             frequenze.get(parola, 0) + 1
16     return frequenze
17
18 testo = "il gatto vede il topo il gatto corre"
19 print(conta_frequenze(testo))
20 # {'il': 3, 'gatto': 2, 'vede': 1,
21 #  'topo': 1, 'corre': 1}
22
23 # 3) Lookup rapido tramite chiave
24 prodotti = {"P001": 19.90, "P002": 7.50}
25 prezzo = prodotti.get("P001", "non trovato")

```

Listato 8: Hash table: rubrica, conteggio frequenze, lookup.

Applicazioni reali: il modello di hash table è alla base dei sistemi di *caching* come Redis [13], e degli *indici* nei motori di database per lookup rapidi su colonne chiave.

VIII. TREE

Un *tree* (albero) organizza i dati secondo una struttura gerarchica: un nodo *root* (radice) da cui discendono nodi *child* (figli), ciascuno collegato al proprio *parent* (genitore) tramite un *edge* (arco); i nodi privi di figli sono detti *leaf* (foglie). La *depth* di un nodo è la distanza dalla radice, l'*height* dell'albero è la profondità massima, e un *subtree* è l'albero radicato in un nodo qualsiasi.

Un *binary tree* è un albero in cui ogni nodo ha al massimo due figli. Un *binary search tree* (BST) è un *binary tree* che rispetta la proprietà d'ordine: per ogni nodo, tutti i valori nel sottoalbero sinistro sono minori e tutti quelli nel sottoalbero destro sono maggiori. Un *balanced tree* (albero bilanciato) è un BST in cui l'altezza è mantenuta $O(\log n)$ tramite rotazioni; esempi notevoli sono l'**AVL tree**, che impone un vincolo rigido sul bilanciamento dei sottoalberi, e il **Red-Black tree**, che usa una colorazione dei nodi per un bilanciamento più lasco ma più economico da mantenere; entrambi garantiscono $O(\log n)$ nel caso peggiore e sono ampiamente usati in implementazioni reali (es. indici di molti database) [1].

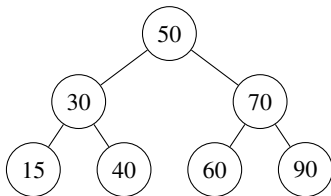


Fig. 6: Struttura gerarchica di un Binary Search Tree.

Il Listato 9 implementa un BST con inserimento, ricerca e i tre attraversamenti classici (Fig. 6).

```

1 class NodoBST:
2     def __init__(self, valore):
3         self.valore = valore
4         self.sinistro = None
5         self.destro = None
6
7 class BST:
8     def __init__(self):

```

```

9         self.root = None
10
11     def insert(self, valore):
12         """Inserimento: O(h), h=altezza."""
13         if self.root is None:
14             self.root = NodoBST(valore)
15         else:
16             self._insert(self.root, valore)
17
18     def _insert(self, nodo, valore):
19         if valore < nodo.valore:
20             if nodo.sinistro is None:
21                 nodo.sinistro = NodoBST(valore)
22             else:
23                 self._insert(nodo.sinistro, valore)
24         else:
25             if nodo.destro is None:
26                 nodo.destro = NodoBST(valore)
27             else:
28                 self._insert(nodo.destro, valore)
29
30     def search(self, valore):
31         """Ricerca: O(h)."""
32         corrente = self.root
33         while corrente:
34             if valore == corrente.valore:
35                 return True
36             corrente = corrente.sinistro if \
37                 valore < corrente.valore else \
38                 corrente.destro
39         return False
40
41     def inorder(self):
42         """Sinistro - Radice - Destro:
43         produce i valori ORDINATI."""
44         risultato = []
45         self._inorder(self.root, risultato)
46         return risultato
47
48     def _inorder(self, nodo, acc):
49         if nodo:
50             self._inorder(nodo.sinistro, acc)
51             acc.append(nodo.valore)
52             self._inorder(nodo.destro, acc)
53
54     def preorder(self):
55         """Radice - Sinistro - Destro."""
56         risultato = []
57         self._preorder(self.root, risultato)
58         return risultato
59
60     def _preorder(self, nodo, acc):
61         if nodo:
62             acc.append(nodo.valore)
63             self._preorder(nodo.sinistro, acc)
64             self._preorder(nodo.destro, acc)
65
66     def postorder(self):
67         """Sinistro - Destro - Radice."""
68         risultato = []
69         self._postorder(self.root, risultato)
70         return risultato
71
72     def _postorder(self, nodo, acc):
73         if nodo:
74             self._postorder(nodo.sinistro, acc)
75             self._postorder(nodo.destro, acc)
76             acc.append(nodo.valore)
77
78 # Esempio di esecuzione
79 albero = BST()
80 for v in [50, 30, 70, 15, 40, 60, 90]:
81     albero.insert(v)
82 print(albero.inorder())
83 # [15, 30, 40, 50, 60, 70, 90] -> ordinati!

```

Listato 9: Binary Search Tree: insert, search, traversal.

Il traversal *inorder* produce sempre gli elementi in ordine crescente perché visita, per ogni nodo, prima l'intero sottoalbero sinistro (valori minori), poi il nodo stesso, poi l'intero sottoalbero destro (valori maggiori).

La complessità delle operazioni fondamentali su un BST

dipende sempre dalla sua altezza h , secondo la relazione:

$$T_{\text{ricerca}} = T_{\text{inserimento}} = T_{\text{cancellazione}} = O(h) \quad (1)$$

In un **BST bilanciato**, $h = O(\log n)$ e quindi ricerca, inserimento e cancellazione costano $O(\log n)$; in un **BST degenerato** (ad esempio ottenuto inserendo valori già ordinati, che produce di fatto una sequenza simile a una lista concatenata), $h = O(n)$ e le stesse operazioni degradano a $O(n)$. Va sottolineato che un BST costruito senza accorgimenti di bilanciamento **non è automaticamente bilanciato**: il bilanciamento è una proprietà che va garantita attivamente (es. tramite rotazioni AVL o Red-Black), non una conseguenza automatica della struttura BST.

Applicazioni reali: gli alberi trovano impiego nell'organizzazione dei *file system* gerarchici e negli *indici* dei database relazionali, tipicamente realizzati con B-tree (una generalizzazione multi-via del BST).

IX. GRAPH

Un *graph* (grafo) generalizza l'albero rimuovendo il vincolo gerarchico: è composto da un insieme di *vertex* (vertici, V) e da un insieme di *edge* (archi, E) che li collegano, senza restrizioni sul numero di connessioni per nodo e senza necessità di una radice. Un grafo è *directed* (diretto) se gli archi hanno un verso, *undirected* altrimenti; è *weighted* (pesato) se a ogni arco è associato un costo, *unweighted* altrimenti. Le due rappresentazioni più comuni sono la *adjacency matrix* (costo di memoria $O(|V|^2)$, test di adiacenza $O(1)$) e la *adjacency list* (costo di memoria $O(|V| + |E|)$, preferibile nei grafi sparsi).

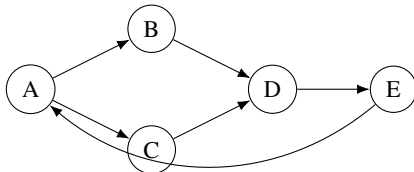


Fig. 7: Grafo diretto con un ciclo ($A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$, $D \rightarrow E$, $E \rightarrow A$).

Il grafo di Fig. 7 contiene deliberatamente un arco di ritorno $E \rightarrow A$, che chiude un ciclo ($A \rightarrow B \rightarrow D \rightarrow E \rightarrow A$): serve a verificare che gli algoritmi di visita presentati di seguito si comportino correttamente anche in presenza di cicli.

A. Visita in Ampiezza (BFS) e in Profondità (DFS)

La *Breadth-First Search* (BFS) utilizza una **queue** come struttura ausiliaria ed esplora il grafo per livelli. La *Depth-First Search* (DFS) utilizza uno **stack** – esplicito oppure implicito tramite la ricorsione e la call stack – e si spinge il più possibile in profondità lungo un ramo prima di tornare indietro (*backtracking*). Con rappresentazione ad adjacency list, entrambi gli algoritmi hanno complessità $O(|V| + |E|)$, dove $|V|$ è il numero di vertici e $|E|$ il numero di archi [1]. La correttezza in presenza di cicli è garantita dall'insieme dei nodi *visitati*, in cui il nodo di partenza viene inserito immediatamente: un nodo già visitato non viene mai riaccodato né rivisitato, per cui l'algoritmo termina sempre anche su grafi ciclici.

```

1 from collections import deque
2
3 grafo = {
4     'A': ['B', 'C'],
5     'B': ['D'],
6     'C': ['D'],
7     'D': ['E'],
8     'E': ['A'],      # arco di ritorno: crea
                        # un ciclo A->B->D->E->A
9 }
10
11 def bfs(grafo, partenza):
12     """Visita in AMPIEZZA: usa una
13     QUEUE. Il nodo di partenza e'
14     marcato visitato subito, quindi
15     l'algoritmo termina anche in
16     presenza di cicli.
17     Costo: O(|V| + |E|)."""
18     visitati = {partenza}
19     coda = deque([partenza])
20     ordine = []
21     while coda:
22         nodo = coda.popleft()
23         ordine.append(nodo)
24         for vicino in grafo[nodo]:
25             if vicino not in visitati:
26                 visitati.add(vicino)
27                 coda.append(vicino)
28     return ordine
29
30 def dfs(grafo, partenza, visitati=None, ordine=None):
31     """Visita in PROFONDITA': usa la
32     call stack (ricorsione). Il nodo
33     di partenza e' marcato visitato
34     subito: corretto anche sui cicli.
35     Costo: O(|V| + |E|)."""
36     if visitati is None:
37         visitati, ordine = set(), []
38     visitati.add(partenza)
39     ordine.append(partenza)
40     for vicino in grafo[partenza]:
41         if vicino not in visitati:
42             dfs(grafo, vicino, visitati, ordine)
43     return ordine
44
45 print("BFS:", bfs(grafo, 'A')) # A B C D E
46 print("DFS:", dfs(grafo, 'A')) # A B D E C

```

Listato 10: Grafo (con ciclo) tramite adjacency list, con BFS e DFS.

La differenza tra BFS e DFS non è solo l'ordine di visita, ma la struttura dati ausiliaria impiegata (queue contro stack) e le proprietà garantite: su un grafo non pesato, BFS trova sempre il cammino con il minor numero di archi tra due nodi, proprietà che DFS non garantisce.

B. Shortest Path e Algoritmo di Dijkstra

Il problema dello *shortest path* a sorgente singola consiste nel determinare, dato un grafo pesato con pesi non negativi, il costo minimo per raggiungere ogni vertice a partire da un vertice sorgente. L'algoritmo di Dijkstra risolve questo problema mantenendo, per ogni vertice, una distanza tentativa dalla sorgente, ed estendendo a ogni passo il vertice non ancora finalizzato con distanza tentativa minima (operazione di *rilasciamento degli archi*, *edge relaxation*) [1]. L'estrazione ripetuta del vertice a distanza minima è esattamente l'operazione che una **coda a priorità** (*priority queue*), realizzabile tramite un **heap** binario, rende efficiente: in Python, il modulo `heapq` fornisce un'implementazione di min-heap su un semplice array [7].

Il Listato 11 implementa Dijkstra su un piccolo grafo pesato.

```

1 import heapq
2 from typing import Dict, List, Tuple

```

```

3 def dijkstra(grafo: Dict[str, List[Tuple[str, int]]],
4             sorgente: str) -> Dict[str, float]:
5     """Distanze minime dalla sorgente,
6     su un grafo pesato con pesi non
7     negativi. Con un min-heap binario:
8      $O((|V| + |E|) \log |V|)$ ."""
9     distanze = {nodo: float('inf') for nodo in grafo}
10    distanze[sorgente] = 0
11    heap = [(0, sorgente)]
12    finalizzati = set()
13
14    while heap:
15        dist_corrente, nodo = heapq.heappop(heap)
16        if nodo in finalizzati:
17            continue # voce obsoleta (lazy deletion)
18        finalizzati.add(nodo)
19        for vicino, peso in grafo.get(nodo, []):
20            nuova_dist = dist_corrente + peso
21            if nuova_dist < distanze[vicino]:
22                distanze[vicino] = nuova_dist
23                heapq.heappush(heap, (nuova_dist,
24                                    vicino))
25    return distanze
26
27 # Grafo pesato: nodo -> [(vicino, peso), ...]
28 grafo_pesato = {
29     'A': [('B', 4), ('C', 1)],
30     'B': [('D', 1)],
31     'C': [('B', 1), ('D', 5)],
32     'D': [('E', 3)],
33     'E': [],
34 }
35
36 print(dijkstra(grafo_pesato, 'A'))
37 # {'A': 0, 'B': 2, 'C': 1, 'D': 3, 'E': 6}

```

Listato 11: Algoritmo di Dijkstra tramite heapq.

Con un min-heap binario, la complessità standard di Dijkstra è $O((|V| + |E|) \log |V|)$: ogni vertice viene estratto dallo heap al più una volta ($O(\log |V|)$ per estrazione) e ogni arco può generare al più un inserimento nello heap ($O(\log |V|)$ per inserimento) [1]. **Applicazione reale:** l'algoritmo di Dijkstra, o sue varianti, è alla base dei *sistemi di navigazione GPS* e degli algoritmi di *routing* nelle reti informatiche, dove i pesi degli archi rappresentano distanze, tempi di percorrenza o costi di trasmissione.

Altre applicazioni del modello a grafo: i *social network* (utenti come vertici, relazioni come archi) e il web stesso, dove pagine e hyperlink formano un grafo diretto alla base dell'algoritmo PageRank [9]; la gestione delle *dipendenze tra pacchetti software*, tipicamente risolta tramite *topological sorting* su un grafo diretto aciclico (DAG).

X. CONFRONTO TRA LE STRUTTURE DATI

La Tabella 4 riassume in forma comparativa la complessità delle operazioni fondamentali per tutte le strutture trattate, contestualizzando ogni valore anziché presentarlo in modo assoluto.

Il confronto evidenzia un principio generale: non esiste una struttura dominante su tutte le operazioni. L'array eccelle nell'accesso indicizzato ma è rigido nell'inserimento arbitrario; la linked list è efficiente nell'inserimento quando il punto di modifica è già noto, ma perde l'accesso diretto per indice; la hash table è eccellente in media ma non offre garanzie nel caso peggiore né mantiene un ordinamento; il BST offre garanzie logaritmiche solo se bilanciato; il grafo ha costi che dipendono sia dalla rappresentazione scelta sia

dall'algoritmo applicato.

XI. APPLICAZIONI REALI

Questa sezione consolida, per ciascuna struttura, almeno due applicazioni reali.

- **Array** → rappresentazione di immagini digitali come matrici di pixel; calcolo numerico vettoriale/matriciale tramite librerie come NumPy [10].
- **Linked List** → gestione dinamica di sequenze di lunghezza non nota a priori; strutture i cui elementi vengono inseriti/rimossi tramite riferimenti diretti anziché per indice.
- **Stack** → call stack dei linguaggi di programmazione e gestione della ricorsione; funzionalità di undo/redo negli editor; parsing delle espressioni; algoritmo DFS.
- **Queue** → scheduling nei sistemi operativi e code di stampa; sistemi di messaggistica come Apache Kafka [11] e RabbitMQ [12] per comunicazione asincrona tra servizi.
- **Hash Table** → sistemi di lookup e cache come Redis [13]; dizionari e strutture chiave-valore in generale.
- **Tree** → file system gerarchici; indici B-tree nei database relazionali; strutture del DOM delle pagine web.
- **Graph** → sistemi di navigazione GPS e routing (Dijkstra); social network e sistemi di raccomandazione basati su relazioni tra entità; gestione delle dipendenze software.

XII. CASO DI STUDIO INTEGRATO

Per mostrare come più strutture dati collaborino all'interno di un unico sistema, si presenta un piccolo progetto Python: un *sistema di gestione delle richieste di un servizio informatico*. La Tabella 5 motiva la scelta di ciascuna struttura.

Il Listato 12 mostra un'implementazione completa e commentata.

```

1 from collections import deque
2 from typing import Dict, List, Optional, Set
3
4 class SistemaRichieste:
5     """Integra Queue, Hash Table, Stack
6     e Graph in un unico sistema."""
7
8     def __init__(self) -> None:
9         self.coda_attesa: deque = deque() # Queue
10        self.info_richieste: Dict[str, dict] = {} # Hash Table
11        self.storico: List[str] = [] # Stack
12        self.dipendenze_servizi: Dict[str, List[str]] = {
13            'auth': ['pagamento'],
14            'pagamento': ['spedizione'],
15            'spedizione': [],
16        }
17        self._prossimo_id = 1
18
19    def nuova_richiesta(self, servizio: str, utente: str) -> str:
20        """Registra e accoda una richiesta.  $O(1)$ 
21        ammortizzato."""
22        rid = f"REQ-{self._prossimo_id}"
23        self._prossimo_id += 1
24        self.info_richieste[rid] = {
25            'servizio': servizio,
26            'utente': utente,
27            'stato': 'in attesa',
28        }
29        self.coda_attesa.append(rid)
30        return rid

```


Tabella 4: Complessità delle principali operazioni.

Struttura	Accesso caratteristico	Ricerca	Inserimento	Rimozione
Array / list	$O(1)$ (per indice)	$O(n)$	$O(1)$ in coda (ammortizzato); $O(n)$ altrove	$O(1)$ in coda; $O(n)$ altrove
Linked List	$O(n)$ (nessun indice diretto)	$O(n)$	$O(1)$ se il riferimento al nodo è già disponibile; $O(n)$ se è necessario individuarlo	$O(1)$ se il riferimento al nodo è già disponibile; $O(n)$ se è necessario individuarlo
Stack	$O(1)$ (solo cima)	$O(n)$	$O(1)$ ammortizzato	$O(1)$
Queue (deque)	$O(1)$ (solo estremità)	$O(n)$	$O(1)$	$O(1)$
Hash Table / dict	–	Average $O(1)$; worst $O(n)$	Average $O(1)$; worst $O(n)$	Average $O(1)$; worst $O(n)$
BST	–	Balanced $O(\log n)$; degenerate $O(n)$	Balanced $O(\log n)$; degenerate $O(n)$	Balanced $O(\log n)$; degenerate $O(n)$
Graph	–	Dipende da rappresentazione e algoritmo (es. $O(V + E)$ per BFS/DFS su adjacency list)	$O(1)$ (adjacency list)	Dipende dalla rappresentazione

Tabella 5: Strutture dati impiegate nel caso di studio.

Componente	Struttura	Motivazione
Richieste in attesa	Queue	Ordine di arrivo (FIFO)
Anagrafica richieste	Hash Table	Lookup per ID, $O(1)$ medio
Storico operazioni	Stack	Annullamento ultima (LIFO)
Dipendenze servizi	Graph	Relazioni causali tra servizi

```

30
31
32 def elabora_prossima(self) -> Optional[str]:
33     """Estrae dalla coda (FIFO), aggiorna lo
34     stato e registra l'operazione nello
35     storico.  $O(1)$ ."""
36     if not self.coda_attesa:
37         return None
38     rid = self.coda_attesa.popleft()
39     self.info_richieste[rid]['stato'] = '
40     completata'
41     self.storico.append(rid)
42     return rid
43
44 def annulla_ultima_operazione(self) -> Optional[
45     str]:
46     """Annulla l'ultima operazione completata
47     (LIFO).  $O(1)$ ."""
48     if not self.storico:
49         return None
50     rid = self.storico.pop()
51     self.info_richieste[rid]['stato'] = '
52     annullata'
53     self.coda_attesa.appendleft(rid)
54     return rid
55
56 def servizi_a_valle(self, servizio: str) -> Set[
57     str]:
58     """Visita BFS del grafo delle dipendenze:
59     quali servizi dipendono, a cascata, da
60     quello indicato.  $O(|V| + |E|)$ ."""
61     visitati: Set[str] = set()
62     coda = deque([servizio])
63     while coda:
64         corrente = coda.popleft()
65         for succ in self.dipendenze_servizi.get(
66             corrente, []):
67             if succ not in visitati:

```

```

63         visitati.add(succ)
64         coda.append(succ)
65     return visitati
66
67 # Esempio di esecuzione
68 sistema = SistemaRichieste()
69 r1 = sistema.nuova_richiesta('auth', 'mario.rossi')
70 r2 = sistema.nuova_richiesta('pagamento', 'anna.
71     bianchi')
72
73 print(sistema.elabora_prossima()) # REQ-1
74 print(sistema.info_richieste['REQ-1'])
75 print(sistema.servizi_a_valle('auth'))
76 # {'pagamento', 'spedizione'}

```

Listato 12: Sistema di gestione delle richieste: Queue + Hash Table + Stack + Graph.

XIII. CRITERI DI SCELTA DELLA STRUTTURA DATI

Non esiste una struttura dati universalmente migliore: la scelta dipende dal tipo di accesso richiesto, dalle operazioni prevalenti, dalla dimensione dei dati, dalla memoria disponibile, dalla necessità di mantenere un ordinamento, dalla frequenza relativa di ricerca rispetto a inserimento e cancellazione, dalla scalabilità attesa e dalla presenza di relazioni complesse tra le entità. Alcuni scenari tipici possono guidare la scelta:

- **Scenario A – molti accessi tramite indice** → Array/list, che garantisce accesso $O(1)$.
- **Scenario B – lookup frequenti tramite chiave** → Hash Table, con costo medio $O(1)$ indipendente dalla dimensione dei dati.
- **Scenario C – relazioni complesse tra entità** → Graph, l'unico modello capace di rappresentare naturalmente connessioni arbitrarie tra elementi.
- **Scenario D – dati organizzati gerarchicamente** → Tree, adatto a rappresentare relazioni di contenimento o discendenza.

- **Scenario E – elaborazione con logica LIFO** → Stack, quando l'ultima operazione registrata deve essere la prima ad essere annullata o processata.
- **Scenario F – elaborazione con logica FIFO** → Queue, quando l'equità nell'ordine di arrivo è un requisito del sistema.

Come mostrato nel caso di studio di Sezione XII, i sistemi informatici reali raramente si affidano a una sola struttura dati: la combinano con altre, ciascuna scelta per rendere efficiente una specifica classe di operazioni.

XIV. STRUTTURE DATI E PYTHON

Python non espone direttamente i modelli teorici discussi, ma fornisce nella libreria standard primitive che ne costituiscono implementazioni concrete, efficienti e collaudate. Va evitata ogni equivalenza assoluta tra modello teorico e tipo nativo: `list` rappresenta il modello di sequenza dinamica, `dict` costituisce un'implementazione basata su hashing del modello di mapping, e così via. La Tabella 6 sintetizza questa corrispondenza senza identificare in modo assoluto teoria e implementazione.

Il Listato 13 mostra un utilizzo minimale di `heapq` come coda a priorità, già impiegata concretamente nell'implementazione di Dijkstra (Listato 11).

```

1 import heapq
2
3 coda_priorita = []
4 heapq.heappush(coda_priorita, (3, "task C"))
5 heapq.heappush(coda_priorita, (1, "task A"))
6 heapq.heappush(coda_priorita, (2, "task B"))
7
8 # Estrazione sempre in ordine di priorità'
9 while coda_priorita:
10     prioritá, task = heapq.heappop(coda_priorita)
11     print(prioritá, task)
12 # 1 task A
13 # 2 task B
14 # 3 task C

```

Listato 13: `heapq`: heap binario utilizzabile come coda a priorità.

XV. ERRORI CONCETTUALI COMUNI

Lo studio delle strutture dati è frequentemente accompagnato da alcuni fraintendimenti ricorrenti:

1. **Confondere struttura dati e algoritmo:** la struttura definisce che cosa è efficiente, l'algoritmo definisce come si raggiunge un risultato usando quelle operazioni.
2. **Confondere Big-O con il caso peggiore:** Big-O è un limite superiore asintotico applicabile a qualunque scenario (migliore, medio, peggiore), che va sempre dichiarato esplicitamente (Sezione II).
3. **Confondere Array e Linked List:** sono entrambe strutture lineari, ma con garanzie di complessità opposte su accesso e inserimento.
4. **Ritenere che la Hash Table sia sempre $O(1)$:** è vero solo in media; il caso peggiore, in presenza di collisioni numerose, è $O(n)$.
5. **Confondere Stack e Queue:** LIFO contro FIFO sono principi opposti.

6. **Confondere Tree e Graph:** ogni albero è un grafo particolare (connesso, aciclico, con una radice), ma non ogni grafo è un albero.
7. **Confondere altezza e profondità di un Tree:** la profondità è una proprietà di un nodo (distanza dalla radice), l'altezza è una proprietà dell'albero (profondità massima).
8. **Usare `list.pop(0)` per implementare una Queue:** introduce un costo $O(n)$ nascosto; la struttura più adatta è `collections.deque`.
9. **Considerare deque una semplice Linked List:** deque è una struttura a doppia estremità con una propria implementazione a blocchi collegati, concettualmente distinta sia da una linked list nodo-per-nodo sia da un array.
10. **Considerare dict una specifica tecnica teorica di collision resolution:** dict è un'implementazione concreta basata su hashing, i cui dettagli interni (ad es. la strategia di gestione delle collisioni) sono specifici di CPython e non vanno generalizzati al modello teorico di hash table.
11. **Dimenticare la space complexity:** valutare un algoritmo solo sulla base del tempo, ignorando la memoria aggiuntiva richiesta (es. dalla ricorsione o da strutture ausiliarie), porta a stime incomplete dell'efficienza.
12. **Assumere che un BST sia sempre bilanciato:** senza un meccanismo esplicito di bilanciamento (AVL, Red-Black), un BST può degenerare fino a $h = O(n)$.
13. **Confondere BFS e DFS:** non differiscono solo nell'ordine di visita, ma nella struttura ausiliaria impiegata (queue contro stack) e nelle proprietà garantite (BFS trova il cammino minimo in numero di archi su grafi non pesati, DFS no).

XVI. CONCLUSIONI

L'analisi condotta mostra che non esiste una struttura dati universalmente migliore: ciascuna rappresenta un compromesso tra tempo di accesso, costo di inserimento e cancellazione, occupazione di memoria e complessità implementativa. La scelta corretta deve essere presa in funzione del problema da risolvere, delle operazioni effettivamente richieste, della complessità computazionale accettabile, della memoria disponibile, della scalabilità attesa e del contesto applicativo specifico. Come mostrato nel caso di studio integrato, i sistemi informatici reali raramente si affidano a una sola struttura dati: la combinano con altre, ciascuna scelta per rendere efficiente una specifica classe di operazioni.

“La scelta della struttura dati è una decisione progettuale che influenza direttamente l'efficienza dell'algoritmo.”

Padroneggiare le sette strutture presentate in questo articolo – e, soprattutto, il criterio con cui sceglierle – costituisce quindi un prerequisito, non un dettaglio accessorio, per la progettazione di software efficiente.

RIFERIMENTI BIBLIOGRAFICI

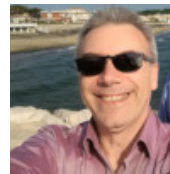
- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA, USA: MIT Press, 2009.

Tabella 6: Corrispondenza tra tipi nativi Python e modelli teorici di riferimento.

Tipo Python	Modello concettuale di riferimento	Operazioni principali	Complessità caratteristica	Utilizzo tipico
<code>list</code>	Sequenza dinamica (dynamic array)	indicizzazione, <code>append</code> , <code>insert</code> , <code>remove</code>	Accesso $O(1)$; <code>append</code> $O(1)$ ammortizzato	Sequenze generiche
<code>tuple</code>	Sequenza immutabile	indicizzazione, iterazione	Accesso $O(1)$	Record fissi, chiavi hashabili
<code>dict</code>	Mapping basato su hashing	<code>get</code> , assegnazione, cancellazione	$O(1)$ medio; $O(n)$ caso peggiore	Lookup per chiave
<code>set</code>	Insieme basato su hashing	<code>add</code> , <code>remove</code> , test di appartenenza	$O(1)$ medio	Deduplicazione, appartenenza
<code>deque</code>	Coda a doppia estremità	<code>append</code> , <code>appendleft</code> , <code>pop</code> , <code>popleft</code>	$O(1)$ su entrambe le estremità	Code, buffer scorrevoli
<code>heapq</code>	Heap binario (min-heap) su array	<code>heappush</code> , <code>heappop</code>	$O(\log n)$ per operazione	Priority queue, Dijkstra

- [2] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Boston, MA, USA: Addison-Wesley Professional, 2011.
- [3] D. E. Knuth, *The Art of Computer Programming, Volume 1: Fundamental Algorithms*, 3rd ed. Boston, MA, USA: Addison-Wesley, 1997.
- [4] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Python*. Hoboken, NJ, USA: Wiley, 2013.
- [5] Python Software Foundation Wiki, “TimeComplexity.” [Online]. Available: <https://wiki.python.org/moin/TimeComplexity>
- [6] Python Software Foundation, “collections — Container datatypes,” Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/collections.html>
- [7] Python Software Foundation, “heapq — Heap queue algorithm,” Python 3 Documentation. [Online]. Available: <https://docs.python.org/3/library/heapq.html>
- [8] Python Software Foundation, “Data Structures,” Python 3 Tutorial. [Online]. Available: <https://docs.python.org/3/tutorial/datastructures.html>
- [9] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank Citation Ranking: Bringing Order to the Web,” Stanford InfoLab, Technical Report 1999-66, 1999.
- [10] NumPy Developers, “NumPy Documentation.” [Online]. Available: <https://numpy.org>
- [11] The Apache Software Foundation, “Apache Kafka.” [Online]. Available: <https://kafka.apache.org>
- [12] RabbitMQ, “Documentation.” [Online]. Available: <https://www.rabbitmq.com>
- [13] Redis Ltd., “Redis Documentation.” [Online]. Available: <https://redis.io>

XVII. PROFILO DELL’AUTORE



ANTONIO ADINOLFI è Docente di Informatica e ingegnere elettronico specializzato in controlli e automazione industriale. Svolge attività di consulenza aziendale nell’ambito della sicurezza nei luoghi di lavoro. I suoi interessi riguardano Intelligenza Artificiale, algoritmi, modellazione matematica, sistemi dinamici, simulazione numerica e programmazione Python, con particolare attenzione all’integrazione tra metodi computazionali, ingegneria e didattica.